CS6843: Program Analysis

Assignment 1: Introduction to LLVM

Roll Number: CS24S009

Problem Statement

The objective of this assignment is to gain hands-on experience with the LLVM infrastructure by installing it and exploring its tools.

1. Sample C Program

```c
#include <stdio.h>

int isCourseAvailableToCredit(int courseNumber);
void runLoop();

int main(){

        printf("Hello LLVM!\n");
        printf("CS6843:Program Analysis\n");
        printf("Assignment 1\n");

        int retVal = isCourseAvailableToCredit(6843);

        if(retVal)
                printf("Course Number Is >5000, Hence Valid !!!\n");

        runLoop();

        return(0);
}

int isCourseAvailableToCredit(int courseNumber){

        if(courseNumber > 5000)
                return 1;
        else
                return 0;
}

void runLoop(){

        for (int i=0; i<10; i++)
                printf("Loop Number: %d\n", i+1);
}
```

2. Commands Used

Generate the human-readable LLVM IR from C source

```
clang -S -emit-llvm HelloLLVM.c -o HelloLLVM.ll
```

Generate the Control Flow Graph (CFG) in GraphViz Dot format

```
opt -passes=dot-cfg HelloLLVM.ll
```

Print the callgraph of the program.

```
opt -passes=print-callgraph HelloLLVM.ll
```

Print loop information.

```
opt -passes='print<loops>' HelloLLVM.ll
```

3. LLVM IR Output

```
; ModuleID = 'HelloLLVM.c'
source_filename = "HelloLLVM.c"
target datalayout = "e-m:e-p270:32:32-p271:32:32-p272:64:64-i64:64-i128:128-f80:128-n8:16:32:64-S128"
target triple = "x86_64-unknown-linux-gnu"

@.str = private unnamed_addr constant [13 x i8] c"Hello LLVM!\0A\00", align 1
@.str.1 = private unnamed_addr constant [25 x i8] c"CS6843:Program Analysis\0A\00", align 1
@.str.2 = private unnamed_addr constant [14 x i8] c"Assignment 1\0A\00", align 1
@.str.3 = private unnamed_addr constant [41 x i8] c"Course Number Is >5000, Hence Valid !!!\0A\00", align 1
@.str.4 = private unnamed_addr constant [17 x i8] c"Loop Number: %d\0A\00", align 1

; Function Attrs: noinline nounwind optnone uwtable
define dso_local i32 @main() #0 {
  %1 = alloca i32, align 4
  %2 = alloca i32, align 4
  store i32 0, ptr %1, align 4
  %3 = call i32 (ptr, ...) @printf(ptr noundef @.str)
  %4 = call i32 (ptr, ...) @printf(ptr noundef @.str.1)
  %5 = call i32 (ptr, ...) @printf(ptr noundef @.str.2)
  %6 = call i32 @isCourseAvailableToCredit(i32 noundef 6843)
  store i32 %6, ptr %2, align 4
  %7 = load i32, ptr %2, align 4
  %8 = icmp ne i32 %7, 0
  br i1 %8, label %9, label %11

9:                                                ; preds = %0
  %10 = call i32 (ptr, ...) @printf(ptr noundef @.str.3)
  br label %11

11:                                               ; preds = %9, %0
  call void @runLoop()
  ret i32 0
}

declare i32 @printf(ptr noundef, ...) #1

; Function Attrs: noinline nounwind optnone uwtable
define dso_local i32 @isCourseAvailableToCredit(i32 noundef %0) #0 {
  %2 = alloca i32, align 4
  %3 = alloca i32, align 4
  store i32 %0, ptr %3, align 4
  %4 = load i32, ptr %3, align 4
  %5 = icmp sgt i32 %4, 5000
  br i1 %5, label %6, label %7

6:                                                ; preds = %1
  store i32 1, ptr %2, align 4
  br label %8

```

```
7:                                                ; preds = %1
  store i32 0, ptr %2, align 4
  br label %8

8:                                                ; preds = %7, %6
  %9 = load i32, ptr %2, align 4
  ret i32 %9
}

; Function Attrs: noinline nounwind optnone uwtable
define dso_local void @runLoop() #0 {
  %1 = alloca i32, align 4
  store i32 0, ptr %1, align 4
  br label %2

2:                                                ; preds = %9, %0
  %3 = load i32, ptr %1, align 4
  %4 = icmp slt i32 %3, 10
  br i1 %4, label %5, label %12

5:                                                ; preds = %2
  %6 = load i32, ptr %1, align 4
  %7 = add nsw i32 %6, 1
  %8 = call i32 (ptr, ...) @printf(ptr noundef @.str.4, i32 noundef %7)
  br label %9

9:                                                ; preds = %5
  %10 = load i32, ptr %1, align 4
  %11 = add nsw i32 %10, 1
  store i32 %11, ptr %1, align 4
  br label %2, !llvm.loop !6

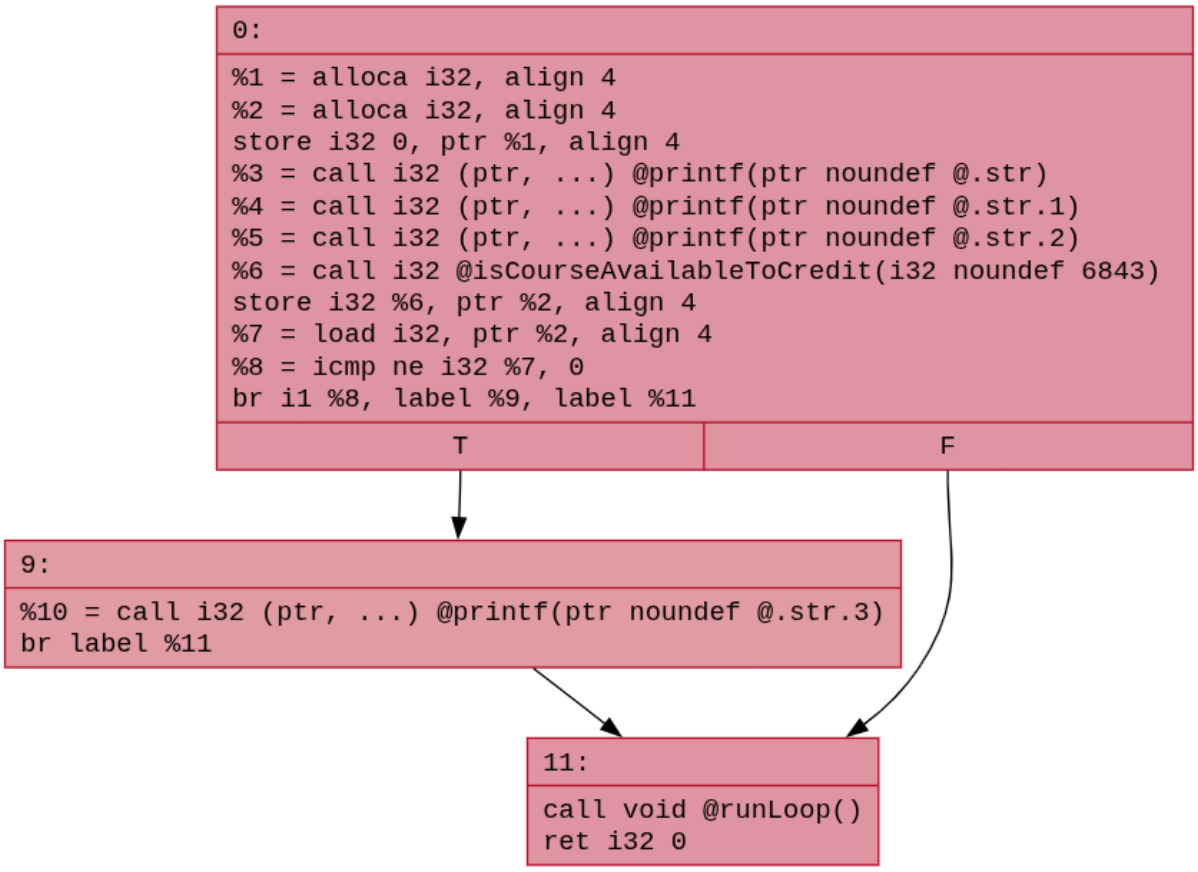
12:                                               ; preds = %2
  ret void
}

attributes #0 = { noinline nounwind optnone uwtable "frame-pointer"="
    all" "min-legal-vector-width"="0" "no-trapping-math"="true" "stack-
    protector-buffer-size"="8" "target-cpu"="x86-64" "target-features"="
    +cmov,+cx8,+fxsr,+mmx,+sse,+sse2,+x87" "tune-cpu"="generic" }
attributes #1 = { "frame-pointer"="all" "no-trapping-math"="true" "
    stack-protector-buffer-size"="8" "target-cpu"="x86-64" "target-
    features"="+cmov,+cx8,+fxsr,+mmx,+sse,+sse2,+x87" "tune-cpu"="
    generic" }

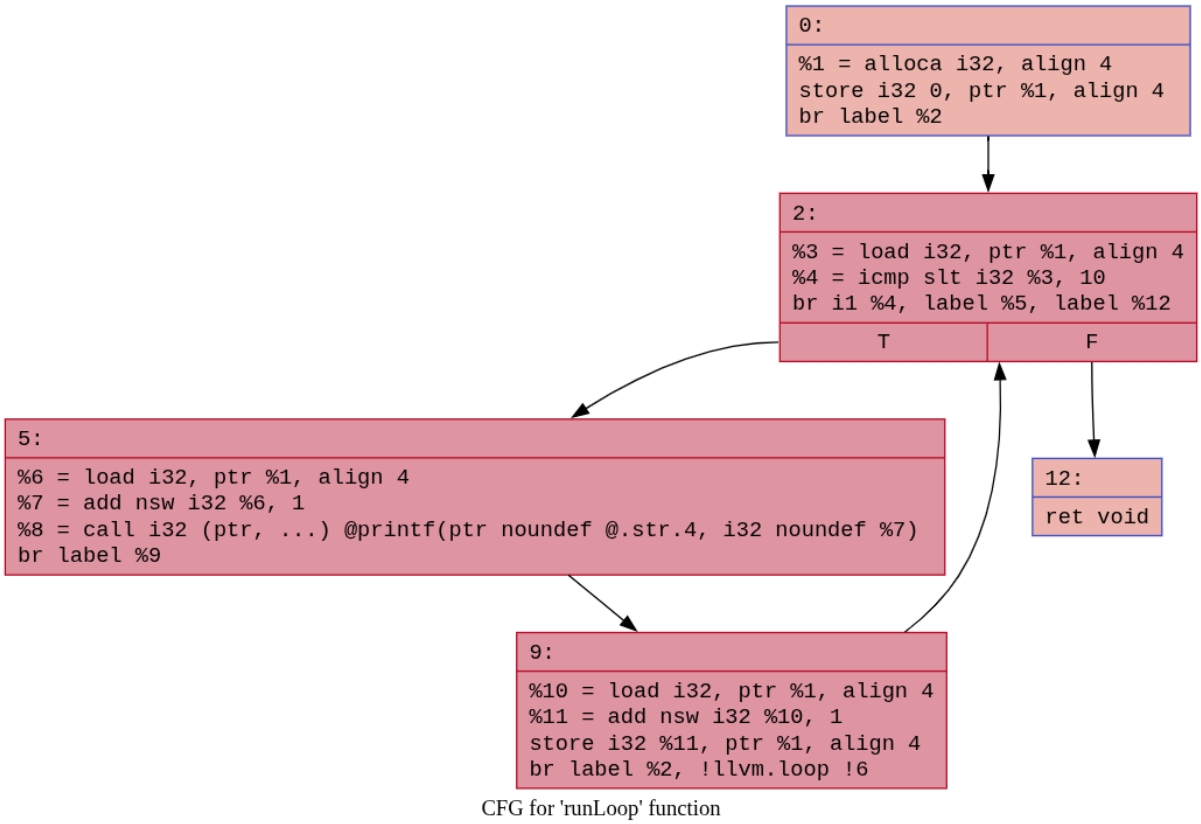
!llvm.module.flags = !{!0, !1, !2, !3, !4}
!llvm.ident = !{!5}

!0 = !{i32 1, !"wchar_size", i32 4}
!1 = !{i32 8, !"PIC Level", i32 2}
!2 = !{i32 7, !"PIE Level", i32 2}
!3 = !{i32 7, !"uwtable", i32 2}
!4 = !{i32 7, !"frame-pointer", i32 2}
!5 = !{!"clang version 20.1.7 (https://github.com/llvm/llvm-project.git
    6146a88f60492b520a36f8f8f3231e15f3cc6082)"}
!6 = distinct !{!6, !7}
!7 = !{!"llvm.loop.mustprogress"}
```

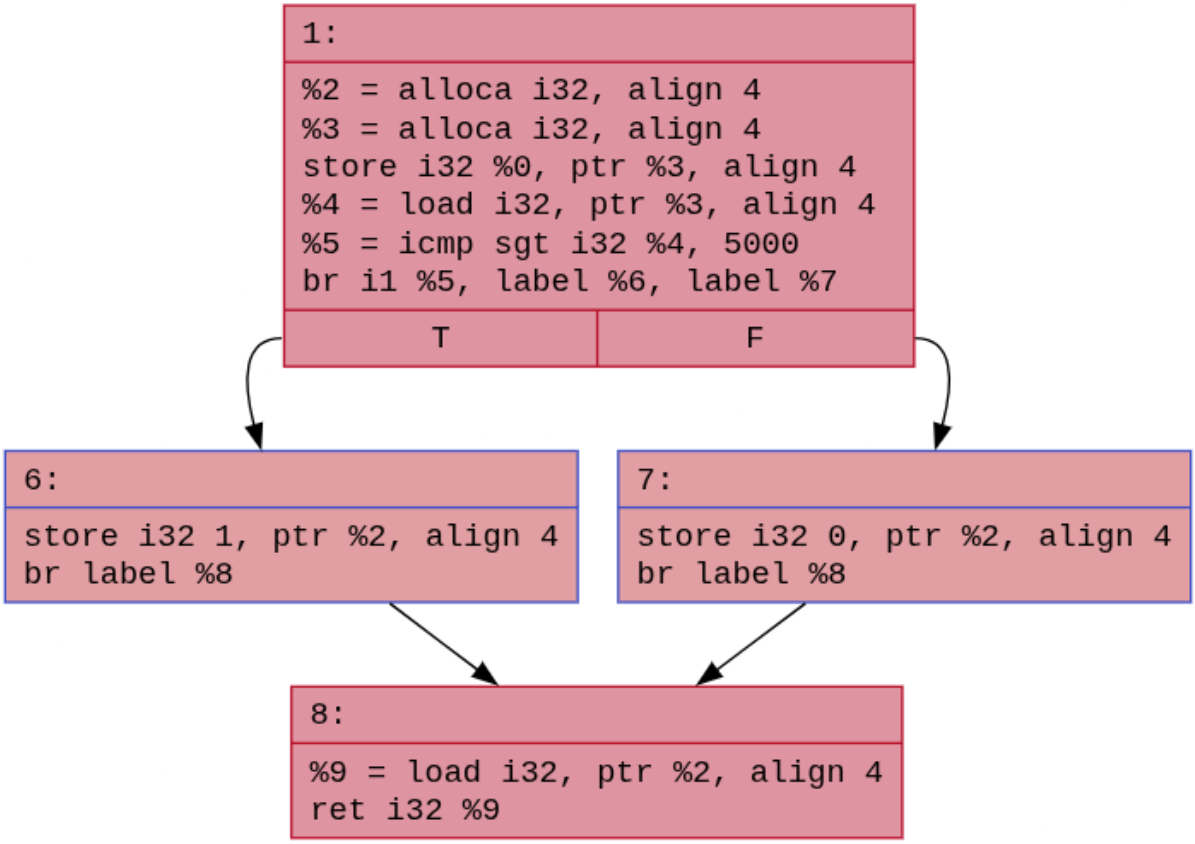
4. Control Flow Graph



CFG for 'main' function

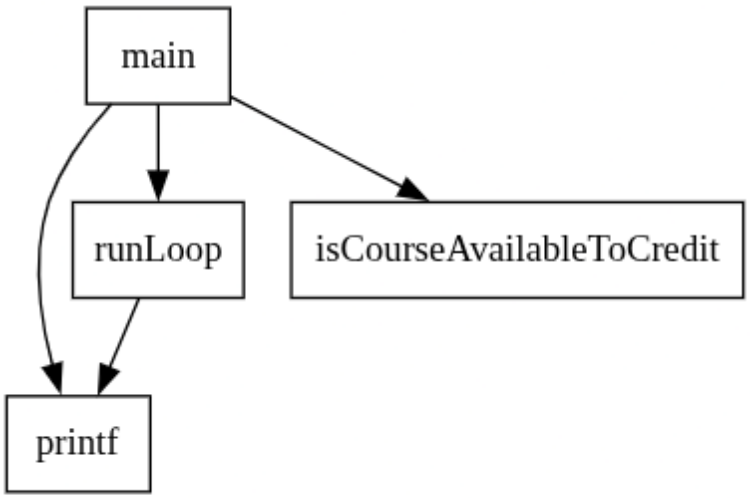


CFG for 'runLoop' function



CFG for 'isCourseAvailableToCredit' function

5. Call Graph



Call graph: HelloLLVM.ll

```
Call graph node <<null function>><<0x64abee02c770>>  #uses=0
  CS<None> calls function 'main'
  CS<None> calls function 'printf'
  CS<None> calls function 'isCourseAvailableToCredit'
  CS<None> calls function 'runLoop'

Call graph node for function:'isCourseAvailableToCredit'<<0x64abee038490>> #uses=2

Call graph node for function: 'main'<<0x64abee037e70>>  #uses=1
  CS<0x64abee029100> calls function 'printf'
  CS<0x64abee0291f0> calls function 'printf'
  CS<0x64abee0292e0> calls function 'printf'
  CS<0x64abee029b50> calls function 'isCourseAvailableToCredit'
  CS<0x64abee029f10> calls function 'printf'
  CS<0x64abee02a130> calls function 'runLoop'

Call graph node for function: 'printf'<<0x64abee037f20>>  #uses=6
  CS<None> calls external node

Call graph node for function: 'runLoop'<<0x64abee038660>>  #uses=2
  CS<0x64abee02b280> calls function 'printf'
```

6. Loop Information Output

```
Loop info for function 'main':
Loop info for function 'isCourseAvailableToCredit':
Loop info for function 'runLoop':
Loop at depth 1 containing: %2<header><exiting>,%5,%9<latch>
```